KÜR · NICHT IM ABITUR · ERMITTLUNG K1

Eigene Datenbank bauen

Projektkarte K1 · CREATE TABLE, INSERT, Fremdschlüssel – und ein Ausblick

LÖSUNG · NUR LEHRKRAFT



> ***FALL-EINSTIEG*** *Juri ist begeistert: „Ich will die Fall-Datenbank selbst bauen, nicht nur benutzen!" Heute legst du Tabellen selbst an, füllst sie und erlebst, wie die Datenbank falsche Verweise abweist. Alles auf diesem Blatt ist **Kür – nicht im Abitur**.*

ERMITTLER-NOTIZ **KÜR – nicht im Abitur**

Im Abitur schreibst du **keine** CREATE- oder INSERT-Anweisungen, und den Verbund schreibst du ausschließlich mit `FROM … WHERE`. Die Schreibweisen mit JOIN auf diesem Blatt sind nur ein Ausblick (nicht im Abitur).

BEWEISSTÜCK 1 **Vom Schema zur Tabelle – die Asservatenkammer**

Die Detektei lagert Beweiskisten in Regalen. Aus jeder Relation wird mit `CREATE TABLE` eine Tabelle, aus jedem Attribut eine Spalte mit **Datentyp** (`INTEGER` = ganze Zahl, `TEXT` = Text). `PRIMARY KEY` kennzeichnet den Primärschlüssel, `FOREIGN KEY (…) REFERENCES …` einen Fremdschlüssel, `NOT NULL` verbietet leere Werte.

`Regal(RegalNr, Raum)`

`Kiste(KisteNr, Inhalt, ↑RegalNr)`

```
CREATE TABLE Regal (
  RegalNr INTEGER PRIMARY KEY,
  Raum    TEXT NOT NULL
);
CREATE TABLE Kiste (
  KisteNr INTEGER PRIMARY KEY,
  Inhalt  TEXT,
  RegalNr INTEGER,
  FOREIGN KEY (RegalNr) REFERENCES Regal(RegalNr)
);
```

Mit `INSERT INTO … VALUES (…)` fügst du Datensätze ein – die Werte in der Reihenfolge der Spalten, Texte in Hochkommata:

```
INSERT INTO Regal VALUES (1, 'Keller');
INSERT INTO Regal VALUES (2, 'Dachboden');
INSERT INTO Regal VALUES (3, 'Archivraum');
INSERT INTO Kiste VALUES (100, 'Fundbuch 2025', 1);
INSERT INTO Kiste VALUES (101, 'Kassenbons Stadtfest', 1);
INSERT INTO Kiste VALUES (102, 'Vereinsakten TuS', 2);
```

BEWEISSTÜCK 2 **Referentielle Integrität – die Datenbank passt auf**

Jeder Fremdschlüsselwert muss auf einen **existierenden** Primärschlüsselwert verweisen (oder NULL sein). Diese Kiste soll in Regal 7 – das es nicht gibt:

```
INSERT INTO Kiste VALUES (103, 'Stadtplan', 7);
```

Die Datenbank meldet `FOREIGN KEY constraint failed` und speichert nichts. (In SQLite muss die Prüfung mit `PRAGMA foreign_keys = ON` eingeschaltet sein – in der Sandbox ist sie es.)

BEWEISSTÜCK 3 Ausblick: andere Schreibweisen für den Verbund (nicht im Abitur)

Abitur-Stil – so schreibst du den Verbund in jeder Prüfung:

```
SELECT k.KisteNr, k.Inhalt, r.Raum
FROM Kiste k, Regal r
WHERE k.RegalNr = r.RegalNr
ORDER BY k.KisteNr;
```

KisteNr	Inhalt	Raum
100	Fundbuch 2025	Keller
101	Kassenbons Stadtfest	Keller
102	Vereinsakten TuS	Dachboden

Kür-Schreibweise mit JOIN … ON – die Verbundbedingung steht hinter ON, das Ergebnis ist gleich:

```
SELECT k.KisteNr, k.Inhalt, r.Raum
FROM Kiste k JOIN Regal r ON k.RegalNr = r.RegalNr
ORDER BY k.KisteNr;
```

LEFT JOIN behält alle Zeilen der linken Tabelle; fehlt der Partner, stehen rechts NULL-Werte. So findest du Regale **ohne** Kiste – das kann der Abitur-Verbund nicht:

```
SELECT r.RegalNr, r.Raum
FROM Regal r LEFT JOIN Kiste k ON r.RegalNr = k.RegalNr
WHERE k.KisteNr IS NULL;
```

RegalNr	Raum
3	Archivraum

Ermittlungsaufträge

NR.	STUFE	ORT	AUFTRAG
1	★☆☆	PAPIER	Die Detektei will ihre Zeuginnen und Zeugen speichern. Notiere die `CREATE TABLE`-Anweisung zu diesem Schema – ZeugeNr ist eine ganze Zahl, Name darf nie leer sein. `Zeuge(ZeugeNr, Name, Telefon)`

LÖSUNG

```
CREATE TABLE Zeuge (
  ZeugeNr INTEGER PRIMARY KEY,
  Name    TEXT NOT NULL,
  Telefon TEXT
);
```

NOT NULL bei Name ist der Zusatz aus dem Auftrag; Telefon darf leer bleiben.

Typische Fehler:

◆ **P9** PRIMARY KEY fehlt oder steht bei Name – dann wären doppelte Nummern erlaubt.

◆ **P5** Texte als Datentyp vergessen; Komma nach der letzten Spalte (Syntaxfehler).

2 ★☆☆ PAPIER

Trage die drei Zeugen mit `INSERT INTO` ein. Noa T. hat kein Telefon.

ZeugeNr	Name	Telefon
1	Mika R.	0000-4711
2	Lena K.	0000-0815
3	Noa T.	*NULL*

LÖSUNG

```
INSERT INTO Zeuge VALUES (1, 'Mika R.', '0000-4711');
```

NR.	STUFE	ORT	AUFTRAG

```
INSERT INTO Zeuge VALUES (2, 'Lena K.', '0000-0815');
INSERT INTO Zeuge VALUES (3, 'Noa T.', NULL);
```

Typische Fehler:

◆ **P5** Name ohne Hochkommata oder `'NULL'` als Text statt NULL.

3 ★★☆ PAPIER

Die Tabelle Vernehmung verweist auf Zeuge. Entscheide für jede Anweisung (nacheinander ausgeführt), ob die Datenbank sie annimmt oder abweist, und begründe.

```
CREATE TABLE Vernehmung (
  VernehmungNr INTEGER PRIMARY KEY,
  ZeugeNr      INTEGER,
  Datum        TEXT,
  FOREIGN KEY (ZeugeNr) REFERENCES Zeuge(ZeugeNr)
);
```

	Anweisung	angenommen? Warum?
a	INSERT INTO Vernehmung VALUES (1, 1, '2026-10-01');	
b	INSERT INTO Vernehmung VALUES (2, 4, '2026-10-02');	
c	INSERT INTO Vernehmung VALUES (1, 2, '2026-10-03');	
d	INSERT INTO Vernehmung VALUES (3, NULL, '2026-10-04');	
e	INSERT INTO Vernehmung VALUES (4, 3, '2026-10-05');	

LÖSUNG

a) angenommen – Zeuge 1 gibt es, VernehmungNr 1 ist neu.

b) abgewiesen – einen Zeugen 4 gibt es nicht; der Fremdschlüssel würde ins Leere zeigen (referentielle Integrität).

c) abgewiesen – VernehmungNr 1 ist schon vergeben (Primärschlüssel doppelt).

d) angenommen – ein Fremdschlüssel darf NULL sein (Zeuge noch unbekannt), solange die Spalte nicht NOT NULL ist.

e) angenommen – Zeuge 3 gibt es.

Ausgeführt (die abgewiesenen Anweisungen fehlen):

```
INSERT INTO Vernehmung VALUES (1, 1, '2026-10-01');
INSERT INTO Vernehmung VALUES (3, NULL, '2026-10-04');
INSERT INTO Vernehmung VALUES (4, 3, '2026-10-05');
```

Diese beiden weist die Datenbank ab (Fremdschlüssel ins Leere bzw. doppelter Primärschlüssel):

```
INSERT INTO Vernehmung VALUES (2, 4, '2026-10-02');
INSERT INTO Vernehmung VALUES (1, 2, '2026-10-03');
```

Typische Fehler:

◆ **P8** „NULL ist verboten" – das gilt nur für Primärschlüssel, nicht für Fremdschlüssel ohne NOT NULL.

4 ★★☆ PAPIER

Ausblick (nicht im Abitur): Schreibe diese Abfrage im Abitur-Stil mit `JOIN … ON` um und gib das Ergebnis nach Auftrag 3 an.

```
SELECT z.Name, v.Datum
FROM Vernehmung v, Zeuge z
WHERE v.ZeugeNr = z.ZeugeNr
```

NR.	STUFE	ORT	AUFTRAG

```
ORDER BY v.Datum;
```

LÖSUNG

```
SELECT z.Name, v.Datum
FROM Vernehmung v JOIN Zeuge z ON v.ZeugeNr = z.ZeugeNr
ORDER BY v.Datum;
```

Name	Datum
Mika R.	2026-10-01
Noa T.	2026-10-05

Vernehmung 3 (ZeugeNr NULL) hat keinen Partner und fällt in beiden Schreibweisen heraus.

Typische Fehler:

◆ P2 Die Verbundbedingung bleibt zusätzlich im WHERE stehen – schadet nicht, ist aber doppelt.

5 ★★★ PAPIER

Ausblick (nicht im Abitur): Juri will wissen, welche Zeugen **noch nie** vernommen wurden. Notiere eine Abfrage mit `LEFT JOIN` und erkläre, warum der Verbund im Abitur-Stil diese Frage nicht beantworten kann.

LÖSUNG

```
SELECT z.ZeugeNr, z.Name
FROM Zeuge z LEFT JOIN Vernehmung v
  ON z.ZeugeNr = v.ZeugeNr
WHERE v.VernehmungNr IS NULL;
```

ZeugeNr	Name
2	Lena K.

Der Verbund mit `FROM … WHERE` liefert nur Paare **mit** Partner; Zeugen ohne Vernehmung tauchen gar nicht auf. Der LEFT JOIN behält sie mit NULL im rechten Teil.

Typische Fehler:

◆ P5 `WHERE v.VernehmungNr = NULL` statt `IS NULL` – liefert nichts.

6 ★★★ LAB

Teamprojekt: Plant eine eigene kleine Datenbank (Projektkarte auf der nächsten Seite) und setzt sie in der Sandbox um.

LÖSUNG

Kriterien siehe Projektkarte; Beispiel einer vollständigen Umsetzung unten auf der Projektkarte.

ABI-ECKE Im Stil des Abiturs (Operator entwerfen, AFB II)

Die folgende Abfrage auf der Fall-Datenbank der Detektei nutzt eine Schreibweise, die im Abitur nicht vorkommt. **Entwerfen** Sie eine gleichwertige Abfrage im Stil der Ergänzenden Hinweise (Verbund über FROM und WHERE). [2 BE]

```
SELECT b.BeweisNr, b.Bezeichnung, o.Bezeichnung AS Fundort
FROM Beweisstueck b JOIN Ort o ON b.OrtNr = o.OrtNr
WHERE o.Stadtteil = 'Hafenviertel'
ORDER BY b.BeweisNr;
```

LÖSUNG

```
SELECT b.BeweisNr, b.Bezeichnung, o.Bezeichnung AS Fundort
FROM Beweisstueck b, Ort o
WHERE b.OrtNr = o.OrtNr
  AND o.Stadtteil = 'Hafenviertel'
ORDER BY b.BeweisNr;
```

BeweisNr	Bezeichnung	Fundort
2	Schloss-Code	Hafenkante
13	Gästebuch	Hafenamt
25	Gästebuch	Hafenamt

1 BE: beide Tabellen in FROM mit Verbundbedingung im WHERE; 1 BE: Stadtteil-Bedingung mit AND, Sortierung übernommen.

Typische Fehler:

◆ P2 Verbundbedingung vergessen – Kreuzprodukt aller Beweisstücke mit allen Orten im Hafenviertel.

SPUR IM LAB Ermittlung K1 „Eigene Datenbank bauen“

Aufgaben: K1-E1-01 … K1-E1-06 (Kür, ohne Rangwirkung) · **Werkzeug:** Sandbox-Konsole im Schreibmodus

Jede Ausführung startet mit einer frischen Datenbank – kaputtmachen ist ausdrücklich erlaubt. Im Lab arbeitest du mit Ermittlern, Fällen und Personen.

COLD CASE · OHNE PUNKTE Das Regal muss weg

Was passiert, wenn du Regal 1 löschen willst, in dem noch Kisten stehen (`DELETE FROM Regal WHERE RegalNr = 1;`)? Probiere es in der Sandbox aus und finde heraus, was `ON DELETE CASCADE` hinter `REFERENCES Regal(RegalNr)` ändern würde.

LÖSUNG

Die Datenbank weist das Löschen ab (FOREIGN KEY constraint failed), weil Kisten 100 und 101 sonst ins Leere verweisen. Mit `ON DELETE CASCADE` würden die Kisten automatisch mitgelöscht – bequem, aber gefährlich (Beweise weg!).

Projektkarte K1 · Unsere Datenbank

Team: _____ Thema: _____

Ideen: Schul-AG, Bandprobe, Tierheim, Sportturnier, Schulbibliothek, Fahrradwerkstatt …

Kernkriterium (wird im Lab geprüft)	erfüllt
mindestens **3 Tabellen**, jede mit Primärschlüssel	
mindestens **eine n:m-Beziehung**: Beziehungstabelle mit zusammengesetztem Primärschlüssel aus zwei Fremdschlüsseln	
jede Tabelle mit **mindestens zwei Datensätzen**	
drei Abfragen im Abitur-Stil (Verbund mit FROM … WHERE, GROUP-BY-Regel)	

LÖSUNG

Beispiel „Schulband" (eine von vielen richtigen Lösungen):

Musiker(MusikerNr, Name, Instrument)

Band(BandNr, Name, Probenraum)

spieltIn(↑MusikerNr, ↑BandNr, Seit)

```
CREATE TABLE Musiker (
  MusikerNr INTEGER PRIMARY KEY,
  Name      TEXT NOT NULL,
  Instrument TEXT
);
CREATE TABLE Band (
  BandNr INTEGER PRIMARY KEY,
  Name   TEXT NOT NULL,
  Probenraum TEXT
);
CREATE TABLE spieltIn (
  MusikerNr INTEGER,
  BandNr    INTEGER,
  Seit      INTEGER,
  PRIMARY KEY (MusikerNr, BandNr),
  FOREIGN KEY (MusikerNr) REFERENCES Musiker(MusikerNr),
  FOREIGN KEY (BandNr) REFERENCES Band(BandNr)
);
INSERT INTO Musiker VALUES (1, 'Mika R.', 'Gitarre');
INSERT INTO Musiker VALUES (2, 'Lena K.', 'Schlagzeug');
INSERT INTO Musiker VALUES (3, 'Noa T.', 'Bass');
INSERT INTO Band VALUES (1, 'Deichgrafen', 'Musikraum A');
INSERT INTO Band VALUES (2, 'Weserwellen', 'Aula');
INSERT INTO spieltIn VALUES (1, 1, 2025);
INSERT INTO spieltIn VALUES (2, 1, 2025);
INSERT INTO spieltIn VALUES (2, 2, 2026);
INSERT INTO spieltIn VALUES (3, 2, 2026);
```

LÖSUNG

Drei Abfragen im Abitur-Stil zum Beispiel „Schulband“:

```
SELECT Name, Instrument
FROM Musiker
WHERE Instrument != 'Gitarre'
ORDER BY Name;
```

Name	Instrument
Lena K.	Schlagzeug
Noa T.	Bass

```
SELECT b.Name, s.Seit
FROM Band b, spieltIn s
WHERE b.BandNr = s.BandNr
  AND s.MusikerNr = 2;
```

Name	Seit
Deichgrafen	2025
Weserwellen	2026

```
SELECT b.BandNr, b.Name, COUNT(*) AS Mitglieder
FROM Band b, spieltIn s, Musiker m
WHERE b.BandNr = s.BandNr
  AND s.MusikerNr = m.MusikerNr
GROUP BY b.BandNr, b.Name
ORDER BY Mitglieder DESC, b.Name;
```

BandNr	Name	Mitglieder
1	Deichgrafen	2
2	Weserwellen	2

Typische Fehler:

◆ **P9** Beziehungstabelle ohne zusammengesetzten Primärschlüssel – dann darf dasselbe Paar doppelt vorkommen.

◆ **P3** Abfrage 3 gruppiert nur nach dem Namen der Band (GROUP-BY-Regel, Gleichnamige).